PREDICT

WEEK 5

Your Function Library

Reusable building blocks

자주 쓰는 함수들을 모아 라이브러리로 만들어요

BIG PICTURE

Small pieces → big builds

Walls, towers, doors — combine them into a house.

작은 함수를 조립해 큰 건물을 만들어요

HOW TO COUNT

Count the axis — one block, one number

1. Stand on the start block — that spot is `(0, 0, 0)`.
2. **x** → walk **right**. Add **+1** for every block.
3. **y** → jump **up**. Add **+1** for every block.
4. **z** → walk **forward**. Add **+1** for every block.

Say them in order every time: `(x, y, z)` = **across, up, forward**.

한 칸 움직이면 +1 — 오른쪽 x, 위로 y, 앞으로 z. 순서는 늘 (x, y, z)

TYPE IT IN

Put the numbers into Minecraft ⌨️

x	y	z
across →	up ↑	forward ⤴

```
place STONE at (x, y, z)
```

- ✓ Open **Code Builder** (press `C` in the world)
- ✓ Type the numbers in order — `(x, y, z)`
- ✓ Press **Run ▶** — your block appears at that spot

코드 빌더(C) 열고 → (x, y, z) 순서로 숫자 넣고 → Run ▶

CONCEPT

A library = many functions

`build_wall(n)`
a wall n long

`build_tower(h)`
a tower h tall

`build_door()`
a doorway

`clear_area(s)`
clear s×s space

각 함수는 한 가지 일을 잘해요

CONCEPT

One reusable building block

```
def build_wall(length):
    for i in range(length):
        agent.place(STONE, FORWARD)
        agent.move(FORWARD, 1)

build_wall(5)
build_wall(3)
```

Call it with any length. 5, then 3 → two walls.

한 함수를 여러 길이로 재사용

CONCEPT

A function can call another

```
def build_house():
    build_wall(4)
    build_tower(3)
    build_door()

build_house()
```

`build_house` reuses the smaller functions.

함수가 다른 함수를 호출해요

WATCH IT!

House = small functions

```
build_house()                                                    →
```

↓ call

```
def build_house():
    build_wall(4)
    build_tower(3)
```

↑ house built

큰 함수가 작은 함수들을 불러요

PREDICT 🤔

How many blocks total?

Predict first — then click reveal

```
def build_wall(length):
    for i in range(length):
        agent.place(STONE, FORWARD)
        agent.move(FORWARD, 1)

build_wall(5)
build_wall(3)
```

Reveal output 🔍 reveal

VOCAB

New words today

`library`	a collection of reusable functions	라이브러리
`reuse`	use the same code again	재사용
`helper`	a small function others call	도우미 함수
`name`	the unique label of a function	이름

DEBUG

What's wrong with this code?

Goal: `build_tower(10)` makes a tower 10 tall — but it stays tiny.

buggy

```
def build_tower(height):
    for i in range(3):
        agent.place(STONE, FORWARD)
        agent.move(UP, 1)

build_tower(10)
```

Look at what `range` uses.

DEBUG

The bug

Bug: the loop uses `range(3)` and ignores `height`. Use the parameter.

fixed

```
def build_tower(height):
    for i in range(height):
        agent.place(STONE, FORWARD)
        agent.move(UP, 1)

build_tower(10)
```

YOUR TURN!

Build with your library 📚

- ✅ Write 2+ functions with clear names
- ✅ Give each its own unique name
- ✅ Make one function call another
- ✅ Build something from the pieces
- ✅ Send a photo of your build

QUIZ

QUESTION 1 OF 3

`build_tower(10)` stays tiny. Why?

- STONE blocks don't stack.
- Functions can't take numbers.
- The loop uses `range(3)` and ignores `height`.
- 10 is too big for a tower.

QUIZ

QUESTION 2 OF 3

Two functions are both named `build_wall`. What's the rule?

- You can only have one function ever.
- Each function needs its own unique name.
- Same names are fine if they're short.
- Functions must all be named `build`.

QUIZ

QUESTION 3 OF 3

Why reuse a function instead of copying code?

- It makes the world bigger.
- Less typing, and you fix bugs in one place.
- Copying is always faster.
- There's no benefit.

QUIZ

Your score

0 / 3

📖 Re-read the slides. You'll get it.

↺ retake

Your toolkit is ready!

Next: a space that reacts.

다음 주: 이벤트 → 확인 → 행동